



CMPE 491 - Senior Design Project I – Fall 2025-2026

High Level Design Report

Team Members:

- Batuhan Taşdemir
- Furkan Cabbar
- Ahmet Emir Ceylan

1. Introduction

The Hybrid Image Enhancement System is designed as a next-generation, device-independent software solution that improves visual quality and zoomed image fidelity while preventing artificial hallucination and over-enhancement. Compared to conventional digital zoom and interpolation-based approaches—where pixel stretching often results in blurred edges, compressed textures, and severe perceptual artifacts—the proposed system introduces a multi-layer architecture driven by explainable and deterministic image-processing logic.

Instead of utilizing AI models as a final, uncontrolled authority, artificial intelligence is only one guided element among several enhancement layers. Each layer of the system is responsible for processing the image at a different abstraction level, including semantic (label-based) enhancement, pixel-level corrections, and global visual optimization. This hybrid strategy enables robust enhancement under varying environmental conditions, device differences, and operational constraints. As documented in the high-level system architecture, the platform is explicitly engineered to remain independent of camera hardware assumptions, minimizing dependence on training datasets and ensuring predictable output across domains.

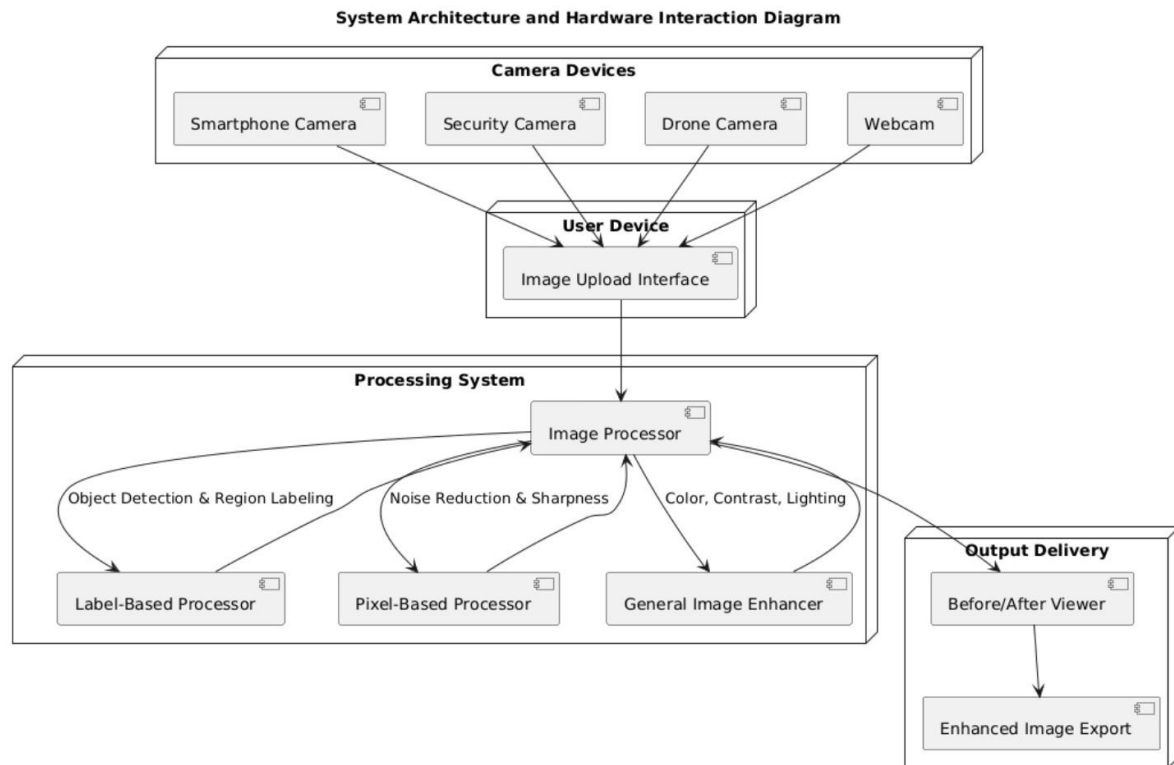
The development of this architecture is driven by three core goals:

- To provide explainable processing, where every enhancement step is deterministic, traceable, and justifiable.
- To deliver device-independent performance by ensuring the same enhancement behavior regardless of sensor source.
- To create modular, maintainable software designed according to separation-of-concerns principles for long-term extensibility and deployment flexibility.

With this foundation, the system is suitable for domains where reliability, traceability, and visual trust are mandatory — such as healthcare imaging, surveillance, legal evidence, document scanning, and industrial inspection.

2. Current System (If Any)

Traditional image zooming and enhancement systems—whether consumer mobile camera software or desktop interpolation-based tools—primarily depend on hardware-level magnification or single-stage interpolation algorithms. These existing solutions generally exhibit several limitations:



Limitation

Description

Interpolation-only enhancement

Linear (bilinear/bicubic) interpolation increases pixel count without generating real detail, resulting in blur and pixelation.

Over-reliance on AI-based hallucination

Recent deep-learning super-resolution models often “invent” non-existent textures, creating inconsistent and untrustworthy details, especially in sensitive use-cases.

Device-dependent behavior

Mobile-based enhancement varies by camera sensor, firmware, or manufacturer pipeline, causing inconsistent results across devices.

Lack of processing explainability

Machine-learning-only systems cannot justify why a detail was generated or enhanced, creating challenges in legal, medical, or industrial imaging.

Global-only enhancement

Current tools often apply sharpen/denoise globally, causing noise amplification, over-contrast, and edge loss.

In contrast, the proposed system eliminates these dependencies by:

- Separating enhancement into subsystems (Profiling, Pixel, Edge, Label-based, Fusion)
- Mapping each enhancement stage to a deterministic service interface
- Using AI models only under constraints and only in semantically significant regions
- Implementing a multi-level architecture (semantic → pixel → global) with orchestration control

As a result, the existing (current) enhancement landscape represents a **single-stage, hardware-tied, interpolation-biased** approach, whereas the proposed system is **layered, explainable, and controllable** — addressing deficiencies at the architectural level before implementation.

3. Proposed Software Architecture

3.1 Overview

We structured the software architecture as a hybrid system that is modular and layered. The goal is to tackle the specific challenges of AI-assisted zooming and image enhancement. Traditional zoom methods often fall short because they rely heavily on simple interpolation or hardware-specific tricks, which means they don't work consistently across different devices or lighting conditions.

Instead of looking at an image as just one flat layer of data, our model breaks it down into three distinct levels of abstraction:

- **Semantic level** (Understanding what is in the image)
- **Pixel level** (Managing the raw data points)
- **Global level** (Adjusting the overall picture)

We handle these levels in a specific sequence. The idea is that each layer fixes problems that the others might miss or even create.

The main reason for building it this way is to stop the common problems we see in AI image processing—like the AI "hallucinating" details that aren't there, over-sharpening things, or struggling with data it hasn't seen before. In this design, Artificial Intelligence isn't the boss making all the decisions. Instead, it acts as a guided tool, working strictly within the boundaries set by classical image processing rules.

3.2 Subsystem Decomposition

To make the system easy to test, maintain, and sustain over time, we followed the "separation of concerns" principle. We broke the architecture down into specific subsystems, where each part has one clear job.

3.2.1 Input and Preprocessing Subsystem

This is the gateway for any data entering the system. Its job is to standardize everything before the real work begins. Since images come from all sorts of cameras, they arrive with different resolutions, color profiles, and noise levels.

At this stage, the system performs a few key checks:

- Verifying that the file is valid and not corrupted.
- Normalizing the image spatially.
- Running a statistical check to measure noise, contrast, brightness, and blur.

The results from this initial scan act as a guide for the rest of the system, telling the later modules which tools to turn on and how strong they should be.

3.2.2 Label-Based Processing Subsystem

Pixels are just numbers; they don't inherently know the difference between a face and a wall. This subsystem solves that by figuring out the semantic "meaning" of the image content so we can treat different areas differently.

There are three steps here: Semantic Feature Extraction

We use deep learning models to spot important parts of the image—edges, text, faces, or structures. We stop treating the image as a grid of dots and start seeing it as a collection of objects.

Mask and Label Map Generation

Once we identify these regions, we create a map (mask). This tells us which pixel belongs to which category, guiding where we need to be careful.

Region-Adaptive Zooming

We don't zoom everything equally. Important areas (like the labeled regions) get high-quality enhancement. Backgrounds or less important spots get a simpler treatment. This prevents the system from inventing fake details in blurry backgrounds while keeping the main subject sharp.

3.2.3 Pixel-Level Processing Subsystem

While the previous layer looked at the "big picture," this layer gets down to the math. Its job is to keep the image physically plausible and numerically stable. Think of it as a cleanup crew that fixes any messes made by the semantic layer.

Here is what happens:

- **Adaptive Noise Suppression:** We clean up random pixel static by looking at the surrounding neighborhood statistics, boosting the signal without killing the texture.

- **Controlled Sharpening:** Zooming often softens an image, so we carefully add sharpness back in. We do this selectively to avoid those ugly white halos (ringing artifacts) around objects.
- **Edge-Preserving Filtering:** We smooth out flat areas but lock down the edges and textures to keep them crisp.

3.2.4 General / Global Image Control Subsystem

This subsystem handles the overall "look" of the image. Unlike the previous steps that work on specific spots, this one fixes global issues like bad lighting or weird color casts from the camera.

We deliberately placed this step **after** the semantic and pixel work. If we did it earlier, we might accidentally crush the details we are trying to save. By doing it last, we ensure the final image looks natural.

Key operations include:

- **Balancing Exposure:** We fix over-bright or too-dark areas to get a nice, even light distribution.
- **Contrast & Dynamic Range:** We adjust the contrast so the image pops but doesn't look blown out or greyed over.
- **Color Correction:** We align the color channels to fix any tint issues, making sure the colors look like they do to the human eye.

3.2.5 Orchestration and Control Subsystem

This is the brain of the operation. Instead of a rigid assembly line where A always leads to B, this controller manages the workflow dynamically.

It handles:

- **Sequencing:** Making sure data flows correctly from Semantic $\rightarrow$ Pixel $\rightarrow$ Global.
- **Error Handling:** If a module crashes, the whole system doesn't die. This controller isolates the bad part and switches to a backup plan (fallback) so the user still gets a result.
- **Real-time Decisions:** Based on that initial scan (noise, blur, etc.), it decides on the fly which modules to run. If an image is too blurry, it might skip aggressive enhancement to avoid making it look fake.

3.3 Hardware / Software Mapping

We built the system to be platform-agnostic. The logical code doesn't care what hardware it's running on.

Execution Modes

- **Client-Side:** Good for privacy or when you have no internet. Runs locally.

- **Server-Side:** Used when we need heavy AI power for big batches or testing.

The Stack

- **Core Language:** Python 3.x
- **AI Training & Modeling:** PyTorch, TensorFlow
- **AI Inference & Optimization:** ONNX Runtime, OpenVINO (for CPU optimization)
- **Computer Vision & Image Processing:** OpenCV, Pillow, scikit-image
- **Scientific Computing & Signal Processing:** NumPy, SciPy
- **User Interface (GUI):** PyQt6 / PySide6
- **Data Visualization:** Matplotlib (for histogram/profiling analysis)
- **Optional Acceleration:** GPU via CUDA (NVIDIA) or ROCm (AMD)

Fallback Strategy:

If the computer doesn't have a fast GPU, the system automatically switches to a CPU-optimized mode. It might be slower, but it won't crash.

3.4 Persistent Data Management

Privacy is the default setting here.

- Processing happens in RAM (memory).
- We wipe temporary buffers immediately.
- Nothing is saved to the disk unless the user says so.

If we do need to save data (for training or testing):

- It is encrypted.
- Names and personal info are stripped out.
- Only authorized people can access it.

3.5 Access Control and Security

We tackle security in layers:

- **User Consent:** You have to click "save" or "share." We don't assume.
- **Sandboxing:** Each process runs in its own box, so a bug in one can't hurt the others.
- **Data Integrity:** We adhere to GDPR principles—only take what you need. AI models are locked and version-controlled so nobody can tamper with them.

3.6 Global Software Control

The control logic uses a "pipeline with branches" model. It is adaptive. The system looks at the situation and decides:

- Which tools to use.
- How hard to push them.

- When to retreat to a safe fallback.

This keeps the software stable, even if things go wrong internally.

3.7 Boundary Conditions

We explicitly planned for the "worst-case scenarios," such as:

- Tiny, low-res inputs.
- Heavy motion blur.
- Corrupted files or bad exposure.

In these cases, the system refuses to hallucinate details. It switches to a conservative mode and honestly tells the user, "This quality is limited." We prioritize reliability and truthfulness over fake prettiness, which is crucial for fields like healthcare or security.

4. Subsystem Services

The architecture exposes specific functional capabilities through discrete service modules. To ensure modularity and ease of integration, data exchange between these components is governed by strict interface contracts.

The primary services include:

- **Image Profiling Service:** Analyzes input statistics to determine processing parameters.
- **Pixel Enhancement Service:** Manages low-level signal corrections and noise reduction.
- **Edge Enhancement Service:** Reconstructs and defines high-frequency structural boundaries.
- **Label-Based Enhancement Service:** Applies semantic-aware processing based on region classification.
- **Fusion and Output Service:** Merges processed layers and finalizes the image for display or storage.

5. Key Terminology

Hybrid Image Enhancement :A methodology that integrates distinct processing techniques—such as deep learning and classical signal processing—into a single pipeline to maximize reliability and image quality.

Device-Independent System :A software design approach that eliminates reliance on specific hardware drivers or sensor characteristics, ensuring consistent behavior across different camera sources.

Explainable Processing: A processing workflow where every modification applied to the image is derived from deterministic, traceable logic, preventing opaque or unpredictable "black box" outcomes.

6. References

- [1] Bruegge, B., & Dutoit, A. H. (2004). *Object-Oriented Software Engineering: Using UML, Patterns, and Java*. Prentice-Hall.
- [2] Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson.
- [3] OpenCV Team. (2025). *OpenCV Image Processing Module Documentation*. Retrieved from <https://docs.opencv.org>
- [4] Bruegge, B., & Dutoit, A. H. (2004). *Object-Oriented Software Engineering: Using UML, Patterns, and Java*. Prentice Hall.
- [5] Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th Edition). Pearson.
- [6] Wang, Z., Bovik, A. C., Sheikh, H. R., & Simoncelli, E. P. (2004). "Image Quality Assessment: From Error Visibility to Structural Similarity (SSIM)." *IEEE Transactions on Image Processing*.
- [7] Zhang, K., Zuo, W., Chen, Y., Meng, D., & Zhang, L. (2017 / 2019). "Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising." *IEEE TIP*.
- [8] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- [9] European Commission (2021). *Ethics Guidelines for Trustworthy AI*.
- [10] ISO/IEC 23091-4:2020. *Image Coding Systems and Quality Evaluation Metrics*.